

PL/SQL EXERCISES

1. CONTROL STRUCTURES

- **Scenario 1: Apply Discount to Senior Customers**

CODE

```
-- ===== -- EXERCISE 1.1: Apply 1% discount to loans for customers over 60 --
=====

DELIMITER //

CREATE PROCEDURE ApplySeniorDiscount() BEGIN DECLARE v_customer_id INT;
DECLARE v_dob DATE; DECLARE v_loan_id INT; DECLARE v_current_rate
DECIMAL(5,2); DECLARE v_age INT; DECLARE done INT DEFAULT FALSE;

-- Declare cursor
DECLARE loan_cursor CURSOR FOR
    SELECT c.CustomerID, c.DOB, l.LoanID, l.InterestRate
    FROM Customers c
    INNER JOIN Loans l ON c.CustomerID = l.CustomerID;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN loan_cursor;

loan_loop: LOOP
    FETCH loan_cursor INTO v_customer_id, v_dob, v_loan_id, v_current_rate;

    IF done THEN
        LEAVE loan_loop;
    END IF;

    -- Calculate age
    SET v_age = TIMESTAMPDIFF(YEAR, v_dob, CURDATE());

    -- Apply 1% discount if age > 60
    IF v_age > 60 THEN
        UPDATE Loans
        SET InterestRate = InterestRate - 1.00
        WHERE LoanID = v_loan_id;
    END IF;
```

```

END LOOP;

CLOSE loan_cursor;

SELECT 'Senior discount applied successfully.' AS Message;

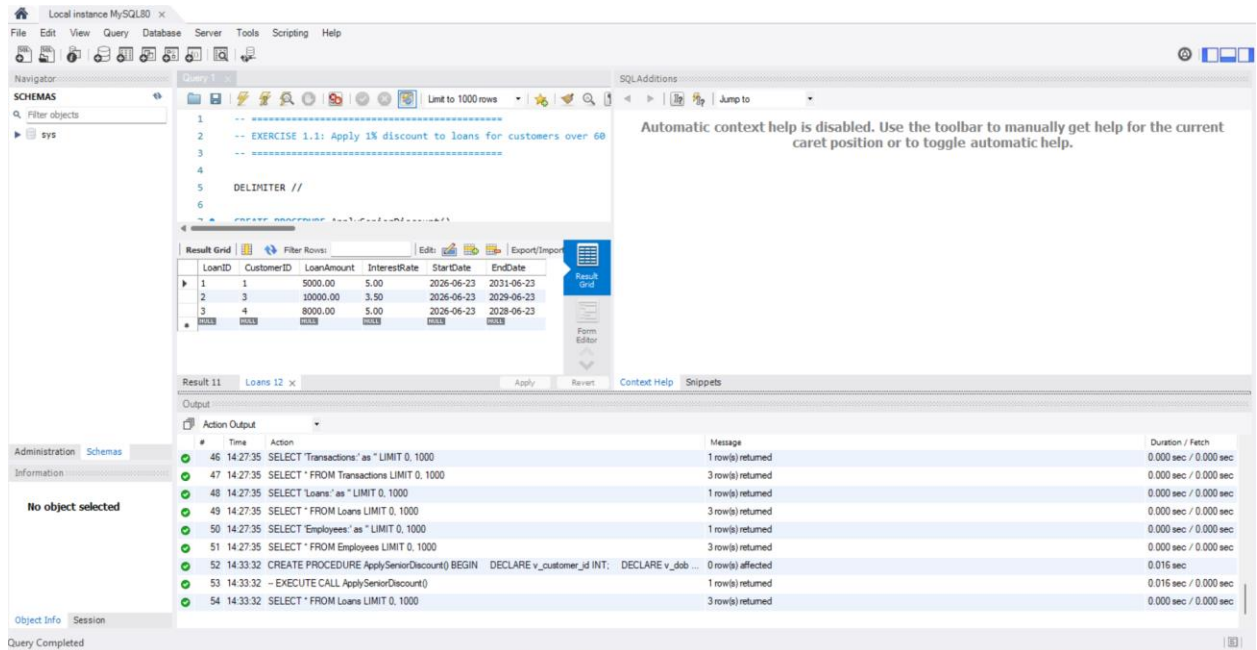
END //

DELIMITER ;

-- EXECUTE CALL ApplySeniorDiscount();

-- Verify the changes SELECT * FROM Loans;

```



- Scenario 2: Update VIP Status **CODE**

```

-- ===== -- EXERCISE 1.2: Update VIP
Status (Balance > $10,000) -- =====

USE bank_db;

DROP PROCEDURE IF EXISTS UpdateVIPStatus;

DELIMITER //

```

```

CREATE PROCEDURE UpdateVIPStatus() BEGIN DECLARE v_customer_id INT;
DECLARE v_balance DECIMAL(15,2); DECLARE done INT DEFAULT FALSE;

DECLARE customer_cursor CURSOR FOR
    SELECT CustomerID, Balance FROM Customers;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN customer_cursor;

customer_loop: LOOP
    FETCH customer_cursor INTO v_customer_id, v_balance;

    IF done THEN
        LEAVE customer_loop;
    END IF;

    IF v_balance > 10000 THEN
        UPDATE Customers
        SET IsVIP = TRUE
        WHERE CustomerID = v_customer_id;
    ELSE
        UPDATE Customers
        SET IsVIP = FALSE
        WHERE CustomerID = v_customer_id;
    END IF;
END LOOP;

CLOSE customer_cursor;

SELECT 'VIP status updated successfully.' AS Message;

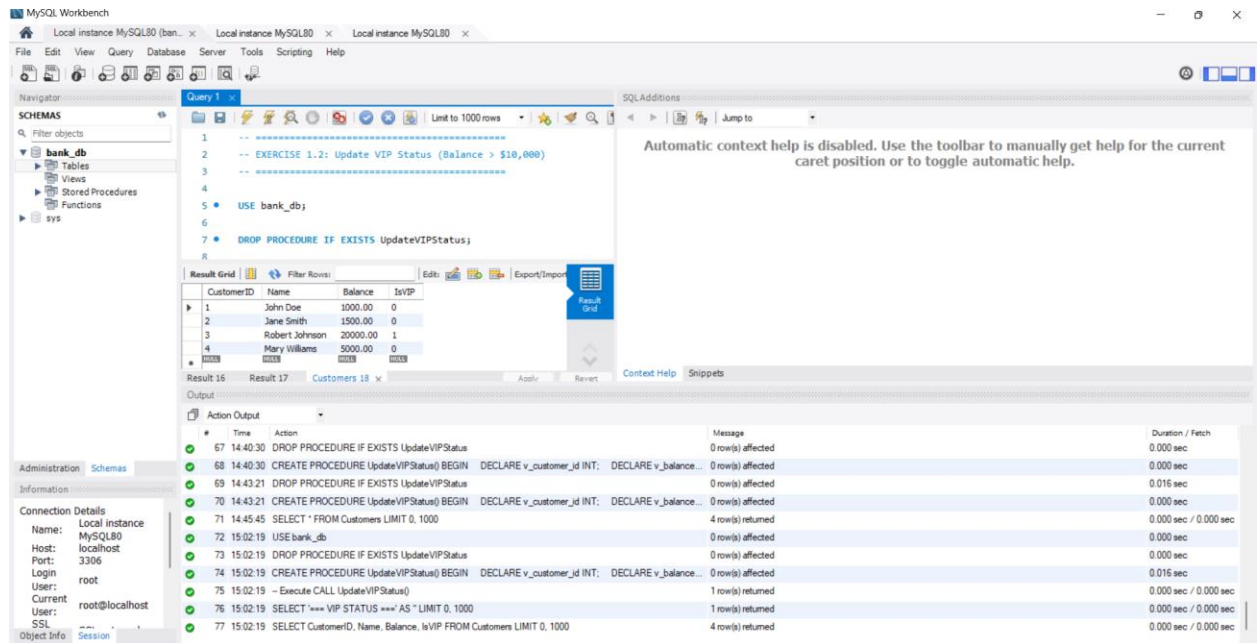
END //

DELIMITER ;

-- Execute CALL UpdateVIPStatus();

-- Verify SELECT '=== VIP STATUS ===' AS "; SELECT CustomerID, Name, Balance, IsVIP
FROM Customers;

```



- Scenario 3: Send Loan Reminders

CODE

```

-- ===== -- EXERCISE 1.3: Send
Reminders for Loans Due in 30 Days --
=====

```

```
USE bank_db;
```

```
DROP PROCEDURE IF EXISTS SendLoanReminders;
```

```
DELIMITER //
```

```
CREATE PROCEDURE SendLoanReminders() BEGIN DECLARE v_customer_name
VARCHAR(100); DECLARE v_loan_id INT; DECLARE v_end_date DATE; DECLARE
v_loan_amount DECIMAL(15,2); DECLARE done INT DEFAULT FALSE;
```

```
DECLARE loan_cursor CURSOR FOR
```

```
    SELECT l.LoanID, c.Name, l.EndDate, l.LoanAmount
    FROM Loans l
```

```
    INNER JOIN Customers c ON l.CustomerID = c.CustomerID
```

```
    WHERE l.EndDate BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL
30 DAY);
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

```
OPEN loan_cursor;
```

```
loan_loop: LOOP
```

```
    FETCH loan_cursor INTO v_loan_id, v_customer_name, v_end_date, v_loan_amount;
```

```
    IF done THEN
```

```
        LEAVE loan_loop;
```

```
    END IF;
```

```
    SELECT CONCAT('REMINDER: Customer ', v_customer_name,  
        ', Loan #', v_loan_id, ' (Amount: $', v_loan_amount,  
        ') is due on ', v_end_date) AS ReminderMessage;
```

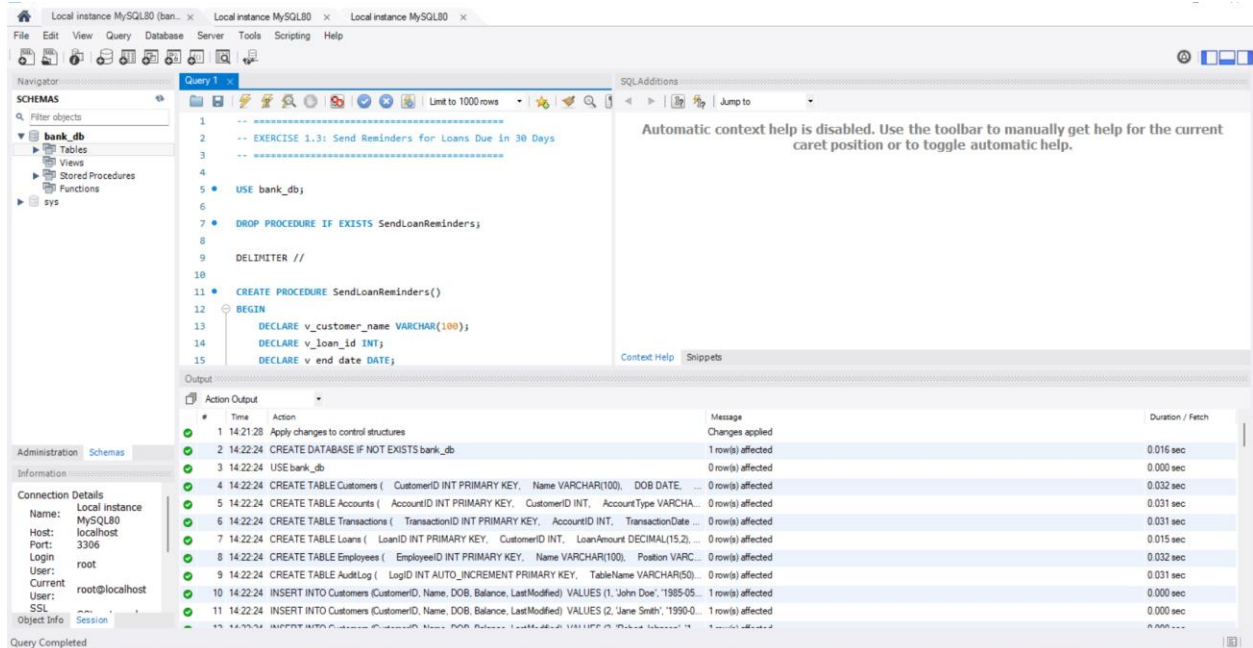
```
END LOOP;
```

```
CLOSE loan_cursor;
```

```
END //
```

```
DELIMITER ;
```

```
-- Execute CALL SendLoanReminders();
```



2. ERROR HANDLING

- **Scenario 1: Safe Transfer Funds**
- CODE**

-- EXERCISE 2.1: Safe Transfer with Error Handling --

```
=====

USE bank_db;

DROP PROCEDURE IF EXISTS SafeTransferFunds;

DELIMITER //

CREATE PROCEDURE SafeTransferFunds( IN p_from_account INT, IN p_to_account INT, IN
p_amount DECIMAL(15,2) ) BEGIN DECLARE v_from_balance DECIMAL(15,2);
DECLARE v_account_exists INT; DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN ROLLBACK; SELECT 'Transaction failed. Error occurred.' AS ErrorMessage; END;

-- Check source account
SELECT COUNT(*) INTO v_account_exists
FROM Accounts
WHERE AccountID = p_from_account;

IF v_account_exists = 0 THEN
    SELECT CONCAT('Source account ', p_from_account, ' does not exist.') AS ErrorMessage;
ELSE
    -- Check destination account
    SELECT COUNT(*) INTO v_account_exists
    FROM Accounts
    WHERE AccountID = p_to_account;

    IF v_account_exists = 0 THEN
        SELECT CONCAT('Destination account ', p_to_account, ' does not exist.') AS
ErrorMessage;
    ELSE
        START TRANSACTION;

        SELECT Balance INTO v_from_balance
        FROM Accounts
        WHERE AccountID = p_from_account;

        IF v_from_balance < p_amount THEN
            ROLLBACK;
            SELECT CONCAT('Insufficient funds. Balance: $', v_from_balance,
                ', Requested: $', p_amount) AS ErrorMessage;
```

```

ELSE
    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;

    INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,
TransactionType)
    VALUES (FLOOR(RAND()*1000000), p_from_account, CURDATE(), p_amount,
'Transfer_Out');

    INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,
TransactionType)
    VALUES (FLOOR(RAND()*1000000), p_to_account, CURDATE(), p_amount,
'Transfer_In');

    COMMIT;
    SELECT CONCAT('Successfully transferred $', p_amount,
        ' from Account ', p_from_account,
        ' to Account ', p_to_account) AS SuccessMessage;
END IF;
END IF;
END IF;

END //

DELIMITER ;

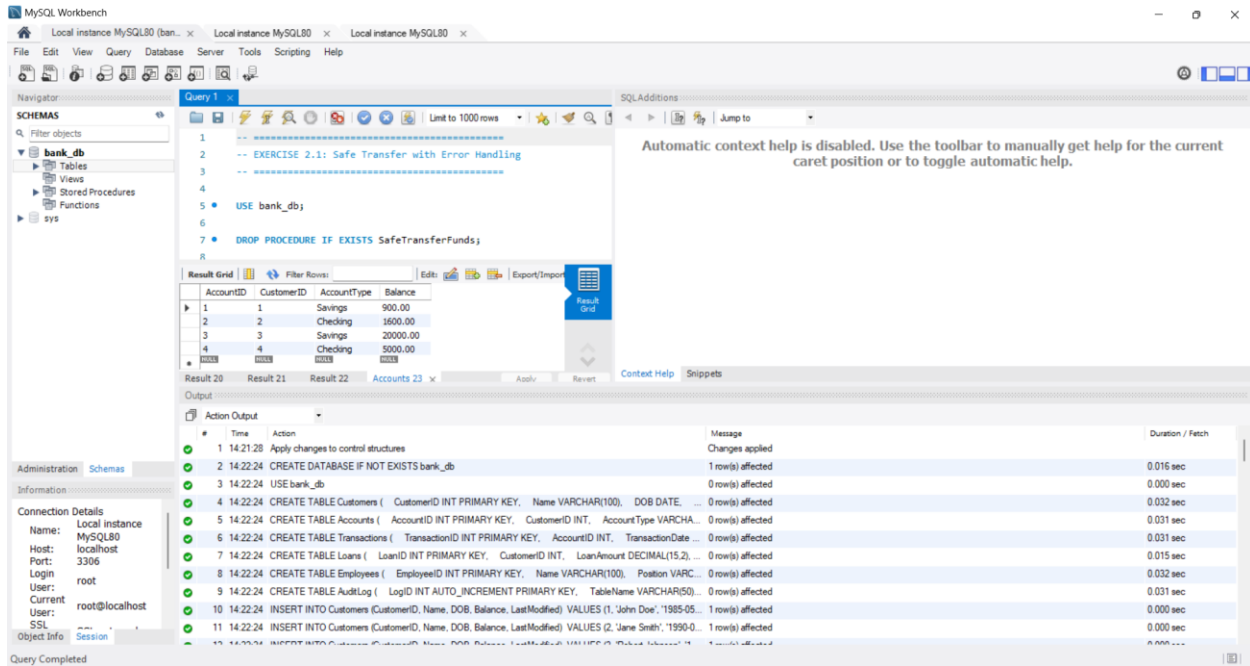
-- Test successful transfer
CALL SafeTransferFunds(1, 2, 100);

-- Test failed transfer (insufficient funds)
CALL SafeTransferFunds(1, 2, 10000);

-- Verify
SELECT '=== ACCOUNT BALANCES ===' AS ";

```

SELECT AccountID, CustomerID, AccountType, Balance FROM Accounts;



- **Scenario 2: Update Salary with Error Handling**

CODE

-- EXERCISE 2.2: Update Employee Salary with Error Handling

-- =====

USE bank_db;

DROP PROCEDURE IF EXISTS UpdateSalary;

DELIMITER //

```
CREATE PROCEDURE UpdateSalary(
IN p_employee_id INT,
  IN p_percentage DECIMAL(5,2)
)
BEGIN
  DECLARE v_exists INT DEFAULT 0;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SELECT 'Error updating salary.' AS ErrorMessage;
```

```

END;

SELECT COUNT(*) INTO v_exists
FROM Employees
WHERE EmployeeID = p_employee_id;

IF v_exists = 0 THEN
    SELECT CONCAT('Employee ID ', p_employee_id, ' does not exist.') AS
ErrorMessage;
ELSE
    START TRANSACTION;
    UPDATE Employees
    SET Salary = Salary * (1 + p_percentage/100)
    WHERE EmployeeID = p_employee_id;
    COMMIT;

    SELECT CONCAT('Salary updated for Employee ID ', p_employee_id,
        '. Increase: ', p_percentage, '%') AS SuccessMessage;
END IF;
END //

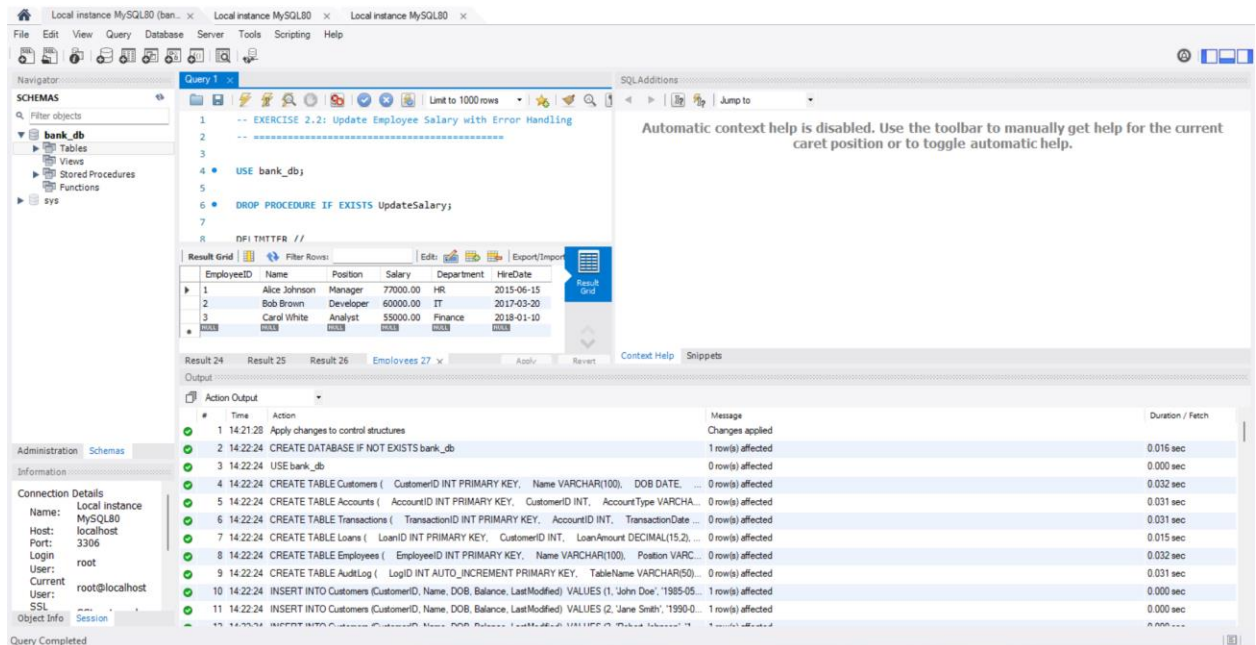
DELIMITER ;

-- Test success
CALL UpdateSalary(1, 10);

-- Test failure (employee doesn't exist)
CALL UpdateSalary(999, 10);

-- Verify
SELECT '=== EMPLOYEE SALARIES ===' AS ";
SELECT * FROM Employees;

```



• Scenario 3: Add customer with Duplicate Check CODE

-- =====
-- EXERCISE 2.3: Add New Customer with Duplicate Check
-- =====

USE bank_db;

DROP PROCEDURE IF EXISTS AddNewCustomer;

DELIMITER //

CREATE PROCEDURE AddNewCustomer(

IN p_customer_id INT,
IN p_name VARCHAR(100),
IN p_dob DATE,
IN p_balance DECIMAL(15,2)
)

BEGIN

DECLARE v_exists INT DEFAULT 0;

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

```

        ROLLBACK;
        SELECT 'Error adding new customer.' AS ErrorMessage;
    END;

    SELECT COUNT(*) INTO v_exists
    FROM Customers
    WHERE CustomerID = p_customer_id;

    IF v_exists > 0 THEN
        SELECT CONCAT('Customer ID ', p_customer_id, ' already exists.') AS
        ErrorMessage;
    ELSE
        START TRANSACTION;
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_customer_id, p_name, p_dob, p_balance, CURDATE());
        COMMIT;

        SELECT CONCAT('Customer ', p_name, ' added successfully.') AS
        SuccessMessage;
    END IF;
END //

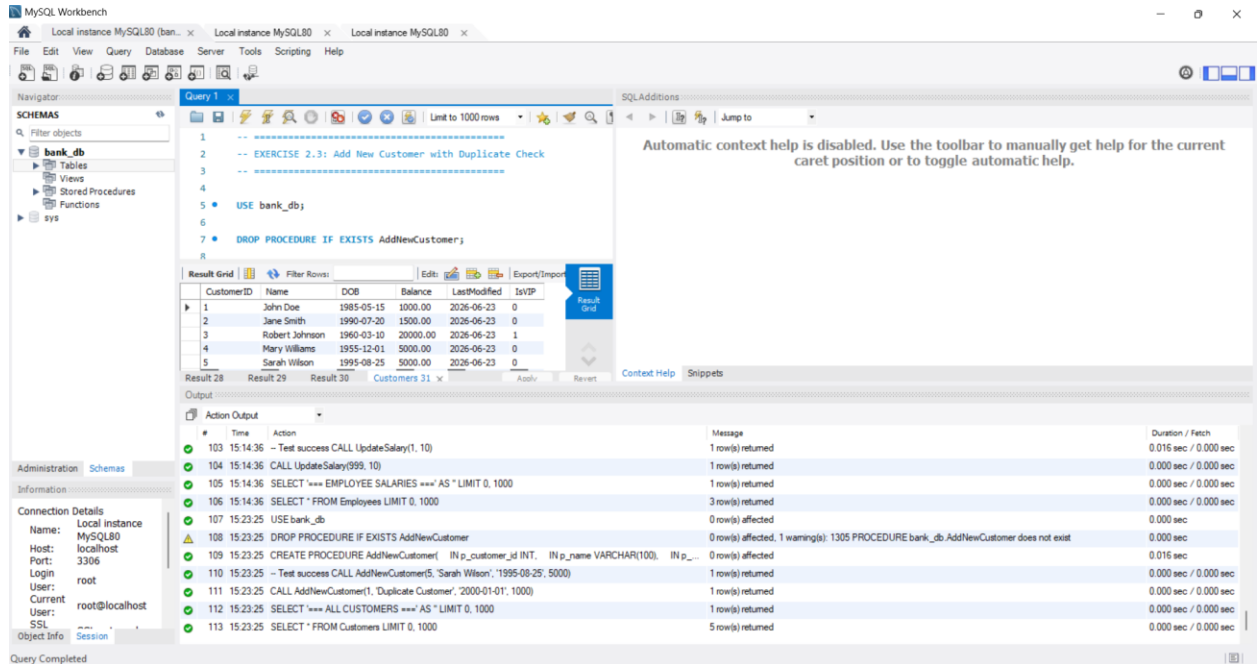
DELIMITER ;

-- Test success
CALL AddNewCustomer(5, 'Sarah Wilson', '1995-08-25', 5000);

-- Test failure (duplicate)
CALL AddNewCustomer(1, 'Duplicate Customer', '2000-01-01', 1000);

-- Verify
SELECT '=== ALL CUSTOMERS ===' AS ";
SELECT * FROM

```



3. Stored Procedures

- **Process Monthly Interest**
CODE

```

-- =====
-- EXERCISE 3.1: Process Monthly Interest for Savings Accounts
-- =====

```

```
USE bank_db;
```

```
DROP PROCEDURE IF EXISTS ProcessMonthlyInterest;
```

```
DELIMITER //
```

```
CREATE PROCEDURE ProcessMonthlyInterest()
```

```
BEGIN
```

```
    DECLARE v_account_id INT;
```

```
    DECLARE v_balance DECIMAL(15,2);
```

```
    DECLARE v_interest DECIMAL(15,2);
```

```
    DECLARE done INT DEFAULT FALSE;
```

```
    DECLARE savings_cursor CURSOR FOR
```

```
        SELECT AccountID, Balance
```

```
        FROM Accounts
```

```

WHERE AccountType = 'Savings';

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN savings_cursor;

savings_loop: LOOP
    FETCH savings_cursor INTO v_account_id, v_balance;

    IF done THEN
        LEAVE savings_loop;
    END IF;

    SET v_interest = v_balance * 0.01;

    UPDATE Accounts
    SET Balance = Balance + v_interest,
        LastModified = CURDATE()
    WHERE AccountID = v_account_id;

    INSERT INTO Transactions (TransactionID, AccountID, TransactionDate,
Amount, TransactionType)
    VALUES (FLOOR(RAND()*1000000), v_account_id, CURDATE(), v_interest,
'Interest');
    END LOOP;

CLOSE savings_cursor;

SELECT 'Monthly interest processed successfully.' AS Message;
END //

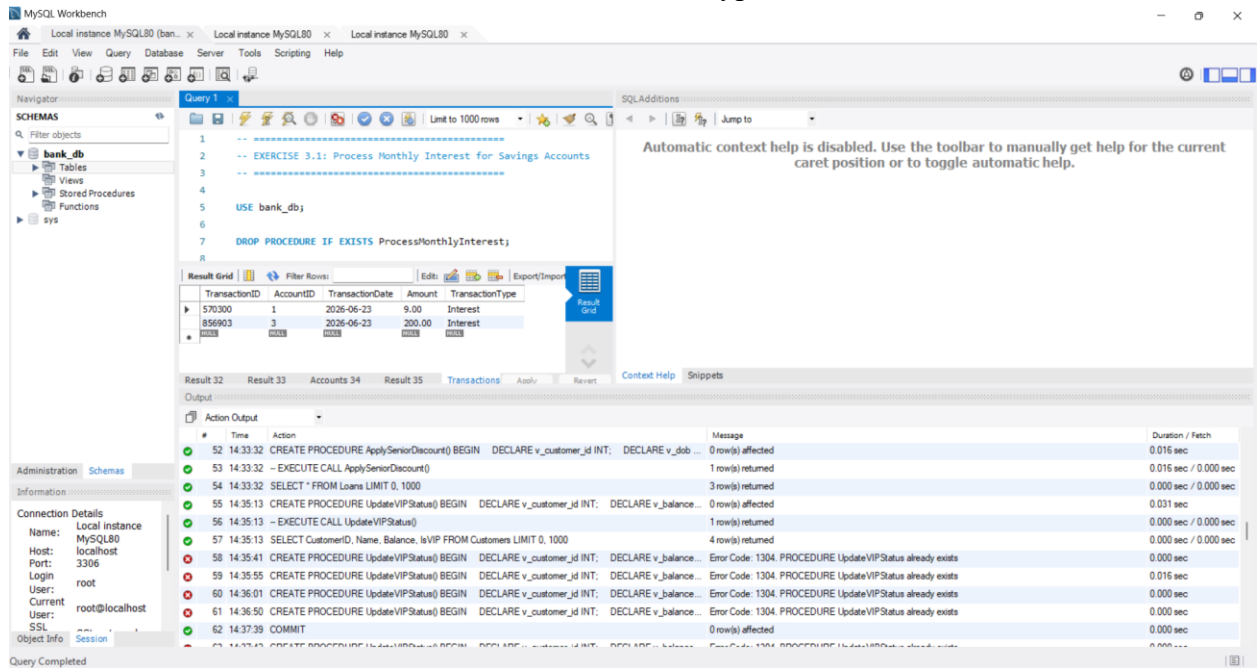
DELIMITER ;

-- Execute
CALL ProcessMonthlyInterest();

-- Verify
SELECT '=== UPDATED SAVINGS ACCOUNTS ===' AS "";
SELECT * FROM Accounts WHERE AccountType = 'Savings';

```

```
SELECT '=== INTEREST TRANSACTIONS ===' AS ";
SELECT * FROM Transactions WHERE TransactionType = 'Interest';
```



• Scenario 2: Update Employee Bonus CODE

```
-- ===== -- EXERCISE 3.2: Update
Employee Bonus by Department --
```

```
=====
```

```
USE bank_db;
```

```
DROP PROCEDURE IF EXISTS UpdateEmployeeBonus;
```

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateEmployeeBonus( IN p_department VARCHAR(50), IN
p_bonus_percentage DECIMAL(5,2) ) BEGIN DECLARE v_count INT;
```

```
SELECT COUNT(*) INTO v_count
```

```
FROM Employees
```

```
WHERE Department = p_department;
```

```
IF v_count = 0 THEN
```

```
    SELECT CONCAT('Department "', p_department, '" has no employees.') AS ErrorMessage;
ELSE
```

```

UPDATE Employees
SET Salary = Salary * (1 + p_bonus_percentage/100)
WHERE Department = p_department;

SELECT CONCAT('Bonus of ', p_bonus_percentage, '% applied to ',
              v_count, ' employees in department ', p_department, '".') AS SuccessMessage;
END IF;

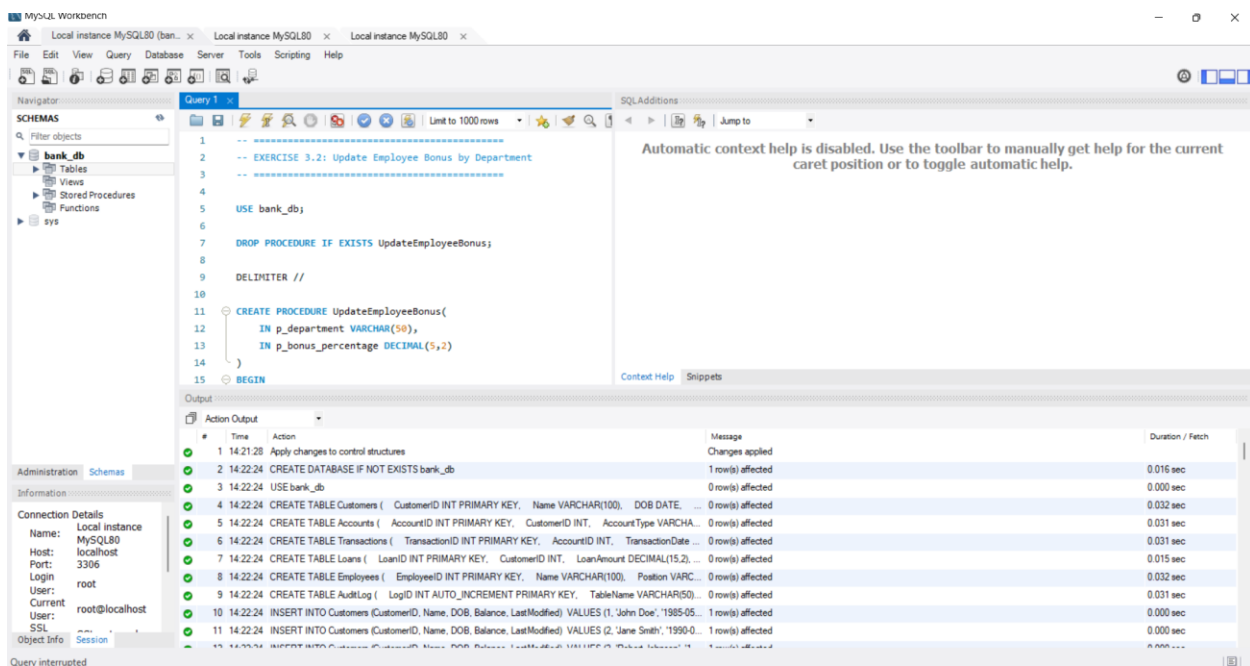
END //

DELIMITER ;

-- Execute CALL UpdateEmployeeBonus('HR', 5); CALL UpdateEmployeeBonus('IT', 10);

-- Verify SELECT '=== EMPLOYEES WITH UPDATED SALARIES ===' AS "; SELECT *
FROM Employees;

```



- Scenario 3: Transfer Funds Between Accounts **CODE**

```

-- ===== -- EXERCISE 3.3: Transfer Funds with Balance Check --
=====

```

```

USE bank_db;

DROP PROCEDURE IF EXISTS TransferFunds;

```

DELIMITER //

```
CREATE PROCEDURE TransferFunds( IN p_from_account INT, IN p_to_account INT, IN
p_amount DECIMAL(15,2) ) BEGIN DECLARE v_from_balance DECIMAL(15,2);
DECLARE v_account_exists INT; DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN ROLLBACK; SELECT 'Transaction failed due to error.' AS ErrorMessage; END;
```

-- Check accounts

```
SELECT COUNT(*) INTO v_account_exists
FROM Accounts
WHERE AccountID = p_from_account;
```

```
IF v_account_exists = 0 THEN
    SELECT CONCAT('Account ', p_from_account, ' does not exist.') AS ErrorMessage;
    RETURN;
END IF;
```

```
SELECT COUNT(*) INTO v_account_exists
FROM Accounts
WHERE AccountID = p_to_account;
```

```
IF v_account_exists = 0 THEN
    SELECT CONCAT('Account ', p_to_account, ' does not exist.') AS ErrorMessage;
    RETURN;
END IF;
```

START TRANSACTION;

```
SELECT Balance INTO v_from_balance
FROM Accounts
WHERE AccountID = p_from_account;
```

```
IF v_from_balance < p_amount THEN
    ROLLBACK;
    SELECT CONCAT('Insufficient funds. Balance: $', v_from_balance,
        ', Requested: $', p_amount) AS ErrorMessage;
ELSE
    UPDATE Accounts
    SET Balance = Balance - p_amount,
        LastModified = CURDATE()
```

```
WHERE AccountID = p_from_account;
```

```
UPDATE Accounts
```

```
SET Balance = Balance + p_amount,
```

```
    LastModified = CURDATE()
```

```
WHERE AccountID = p_to_account;
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,  
TransactionType)
```

```
VALUES (FLOOR(RAND()*1000000), p_from_account, CURDATE(), p_amount,  
'Transfer_Out');
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,  
TransactionType)
```

```
VALUES (FLOOR(RAND()*1000000), p_to_account, CURDATE(), p_amount,  
'Transfer_In');
```

```
COMMIT;
```

```
SELECT CONCAT('Successfully transferred $', p_amount,
```

```
    ' from account ', p_from_account,
```

```
    ' to account ', p_to_account) AS SuccessMessage;
```

```
END IF;
```

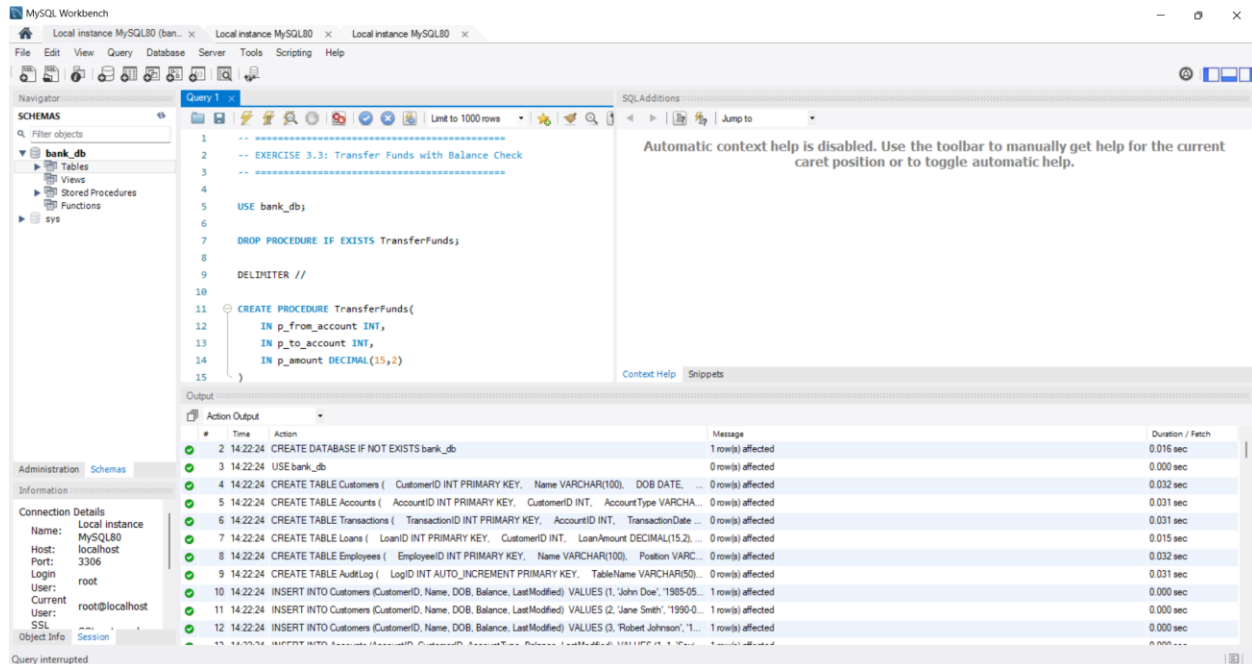
```
END //
```

```
DELIMITER ;
```

```
-- Execute CALL TransferFunds(1, 2, 100);
```

```
-- Verify SELECT '=== ACCOUNT BALANCES ===' AS "; SELECT * FROM Accounts;
```

```
SELECT '=== TRANSFER TRANSACTIONS ===' AS "; SELECT * FROM Transactions  
WHERE TransactionType LIKE 'Transfer%';
```



4. FUNCTIONS

- Scenario 1: Calculate Age Function

CODE

```

-- =====
-- EXERCISE 4.1: Calculate Customer Age
-- =====

```

```
USE bank_db;
```

```
DROP FUNCTION IF EXISTS CalculateAge;
```

```
DELIMITER //
```

```
CREATE FUNCTION CalculateAge(p_dob DATE)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    RETURN TIMESTAMPTDIFF(YEAR, p_dob, CURDATE());
```

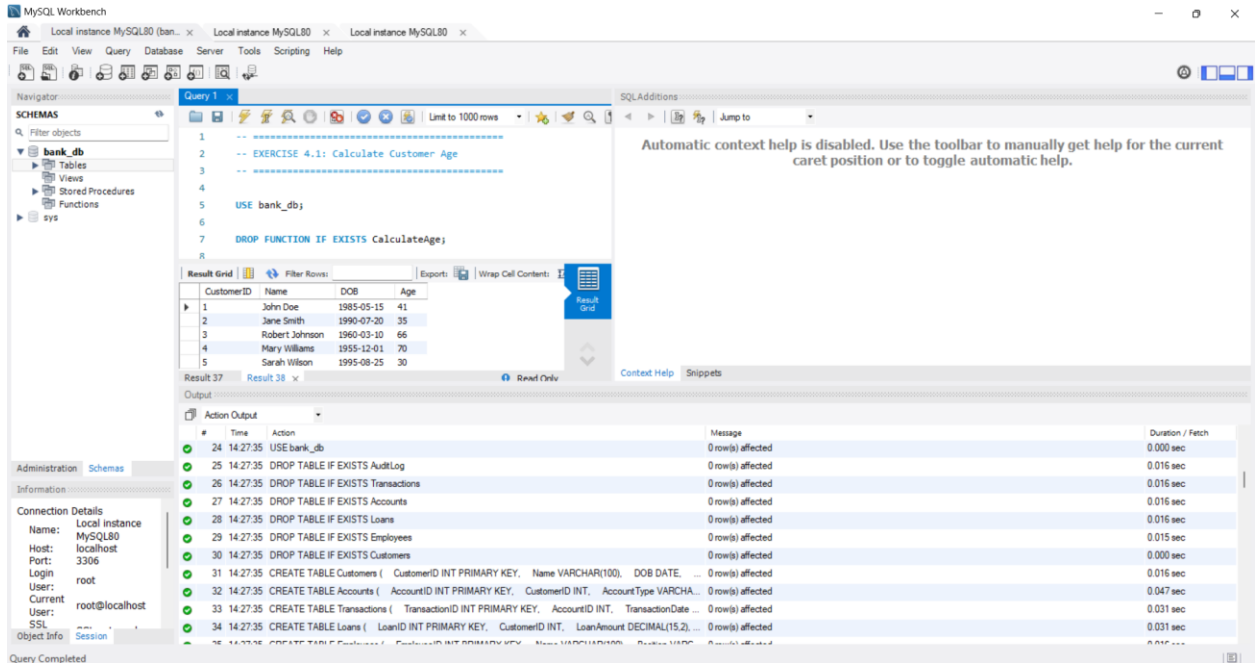
```
END //
```

```
DELIMITER ;
```

-- Test

```
SELECT '=== CUSTOMER AGES ===' AS ";
```

```
SELECT CustomerID, Name, DOB, CalculateAge(DOB) AS Age FROM Customers;
```



• Scenario 2: Calculate Monthly Installment CODE

```
-- =====
-- EXERCISE 4.2: Calculate Monthly Loan Installment
-- =====
```

```
USE bank_db;
```

```
DROP FUNCTION IF EXISTS CalculateMonthlyInstallment;
```

```
DELIMITER //
```

```
CREATE FUNCTION CalculateMonthlyInstallment(
    p_loan_amount DECIMAL(15,2),
    p_interest_rate DECIMAL(5,2),
    p_duration_years INT
)
RETURNS DECIMAL(15,2)
```

DETERMINISTIC

BEGIN

DECLARE v_monthly_rate DECIMAL(15,10);

DECLARE v_months INT;

DECLARE v_installment DECIMAL(15,2);

IF p_interest_rate = 0 OR p_duration_years = 0 THEN

RETURN ROUND(p_loan_amount / (p_duration_years * 12), 2);

END IF;

SET v_monthly_rate = p_interest_rate / 100 / 12;

SET v_months = p_duration_years * 12;

SET v_installment = p_loan_amount * v_monthly_rate *

POW(1 + v_monthly_rate, v_months) /

(POW(1 + v_monthly_rate, v_months) - 1);

RETURN ROUND(v_installment, 2);

END //

DELIMITER ;

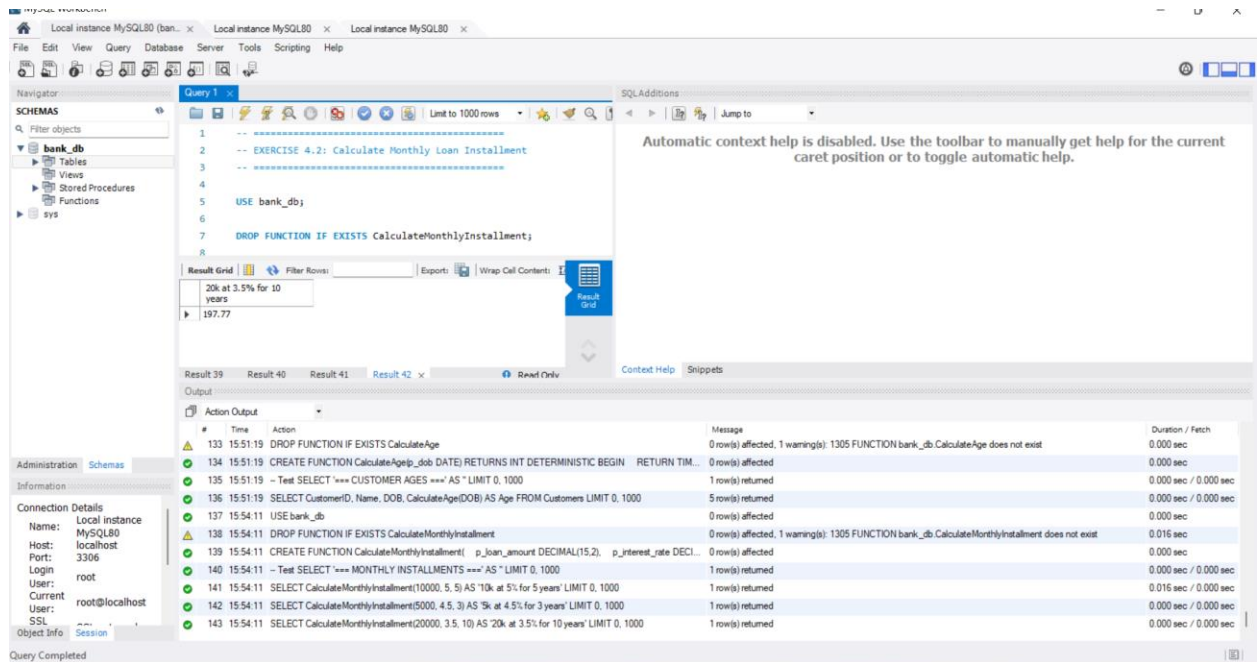
-- Test

SELECT '=== MONTHLY INSTALLMENTS ===' AS '';

SELECT CalculateMonthlyInstallment(10000, 5, 5) AS '10k at 5% for 5 years';

SELECT CalculateMonthlyInstallment(5000, 4.5, 3) AS '5k at 4.5% for 3 years';

SELECT CalculateMonthlyInstallment(20000, 3.5, 10) AS '20k at 3.5% for 10 years';



• Scenario 3: Check Sufficient Balance CODE

```

-- =====
-- EXERCISE 4.3: Check Sufficient Balance
-- =====

```

```
USE bank_db;
```

```
DROP FUNCTION IF EXISTS HasSufficientBalance;
```

```
DELIMITER //
```

```

CREATE FUNCTION HasSufficientBalance(
    p_account_id INT,
    p_amount DECIMAL(15,2)
)
RETURNS BOOLEAN
DETERMINISTIC
BEGIN

```

```

    DECLARE v_balance DECIMAL(15,2);
    DECLARE v_exists INT;

```

```

    SELECT COUNT(*) INTO v_exists
    FROM Accounts

```

```
WHERE AccountID = p_account_id;
```

```
IF v_exists = 0 THEN
```

```
    RETURN FALSE;
```

```
END IF;
```

```
SELECT Balance INTO v_balance
```

```
FROM Accounts
```

```
WHERE AccountID = p_account_id;
```

```
RETURN v_balance >= p_amount;
```

```
END //
```

```
DELIMITER ;
```

```
-- Test
```

```
SELECT '=== BALANCE CHECKS ===' AS '';
```

```
SELECT HasSufficientBalance(1, 500) AS 'Account 1 has $500?';
```

```
SELECT HasSufficientBalance(1, 1500) AS 'Account 1 has $1500?';
```

```
SELECT HasSufficientBalance(3, 5000) AS 'Account 3 has $5000?';
```

```
SELECT HasSufficientBalance(999, 100) AS 'Account 999 has $100? (Should be FALSE)';
```

The screenshot displays the MySQL Workbench interface. The 'Query' window shows a script for Exercise 4.3: Check Sufficient Balance. The script includes a USE statement for the bank_db database, a DROP FUNCTION IF EXISTS statement for HasSufficientBalance, and a CREATE FUNCTION statement. The function takes p_account_id INT and p_amount DECIMAL(15,2) as inputs and returns a boolean. The function body includes a comment and a SELECT statement to check the account balance. The 'Output' window shows the results of the query execution, including the function definition and the results of the test queries. The 'Action Output' window shows the execution details for each query, including the time taken and the number of rows affected.

#	Time	Action	Message	Duration / Fetch
141	15:54:11	SELECT CalculateMonthlyInstallment(10000, 5, 5) AS '10k at 5% for 5 years' LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
142	15:54:11	SELECT CalculateMonthlyInstallment(5000, 4.5, 3) AS '5k at 4.5% for 3 years' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
143	15:54:11	SELECT CalculateMonthlyInstallment(20000, 3.5, 10) AS '20k at 3.5% for 10 years' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
144	15:57:44	USE bank_db;	0 row(s) affected	0.000 sec
145	15:57:44	DROP FUNCTION IF EXISTS HasSufficientBalance;	0 row(s) affected, 1 warning(s): 1305 FUNCTION bank_db.HasSufficientBalance does not exist	0.000 sec
146	15:57:44	CREATE FUNCTION HasSufficientBalance(p_account_id INT, p_amount DECIMAL(15,2)) RETURNS ...	0 row(s) affected	0.000 sec
147	15:57:44	-- Test SELECT '=== BALANCE CHECKS ===' AS '' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
148	15:57:44	SELECT HasSufficientBalance(1, 500) AS 'Account 1 has \$500?' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
149	15:57:44	SELECT HasSufficientBalance(1, 1500) AS 'Account 1 has \$1500?' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
150	15:57:44	SELECT HasSufficientBalance(3, 5000) AS 'Account 3 has \$5000?' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
151	15:57:44	SELECT HasSufficientBalance(999, 100) AS 'Account 999 has \$100? (Should be FALSE)' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

5. TRIGGERS

- **Scenario 1: Update Last Modified Date**

CODE

```
-- =====
-- EXERCISE 5.1: Auto-Update LastModified on Customer Update
-- =====

USE bank_db;

DROP TRIGGER IF EXISTS UpdateCustomerLastModified;

DELIMITER //

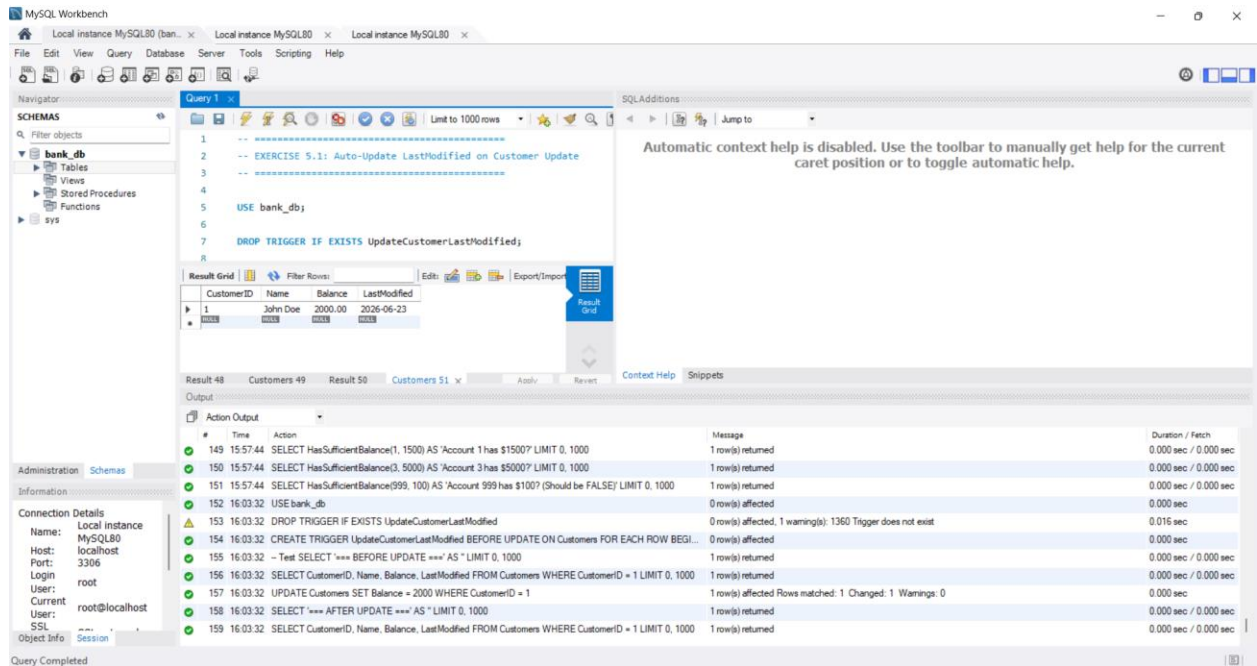
CREATE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    SET NEW.LastModified = CURDATE();
END //

DELIMITER ;

-- Test
SELECT '=== BEFORE UPDATE ===' AS "";
SELECT CustomerID, Name, Balance, LastModified FROM Customers WHERE
CustomerID = 1;

UPDATE Customers SET Balance = 2000 WHERE CustomerID = 1;

SELECT '=== AFTER UPDATE ===' AS "";
SELECT CustomerID, Name, Balance, LastModified FROM Customers WHERE
CustomerID = 1;
```



• Scenario 2: Audit Log Trigger CODE

```

-- =====
-- EXERCISE 5.2: Audit Log for Transactions
-- =====

```

```
USE bank_db;
```

```
DROP TRIGGER IF EXISTS LogTransaction;
```

```
DELIMITER //
```

```
CREATE TRIGGER LogTransaction
```

```
AFTER INSERT ON Transactions
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO AuditLog (TableName, ActionType, RecordID, ActionDate,  
Description)
```

```
    VALUES ('Transactions', 'INSERT', NEW.TransactionID, NOW(),
```

```
            CONCAT('Transaction of $', NEW.Amount, ' (', NEW.TransactionType,  
                ') on account ', NEW.AccountID));
```

```
END //
```

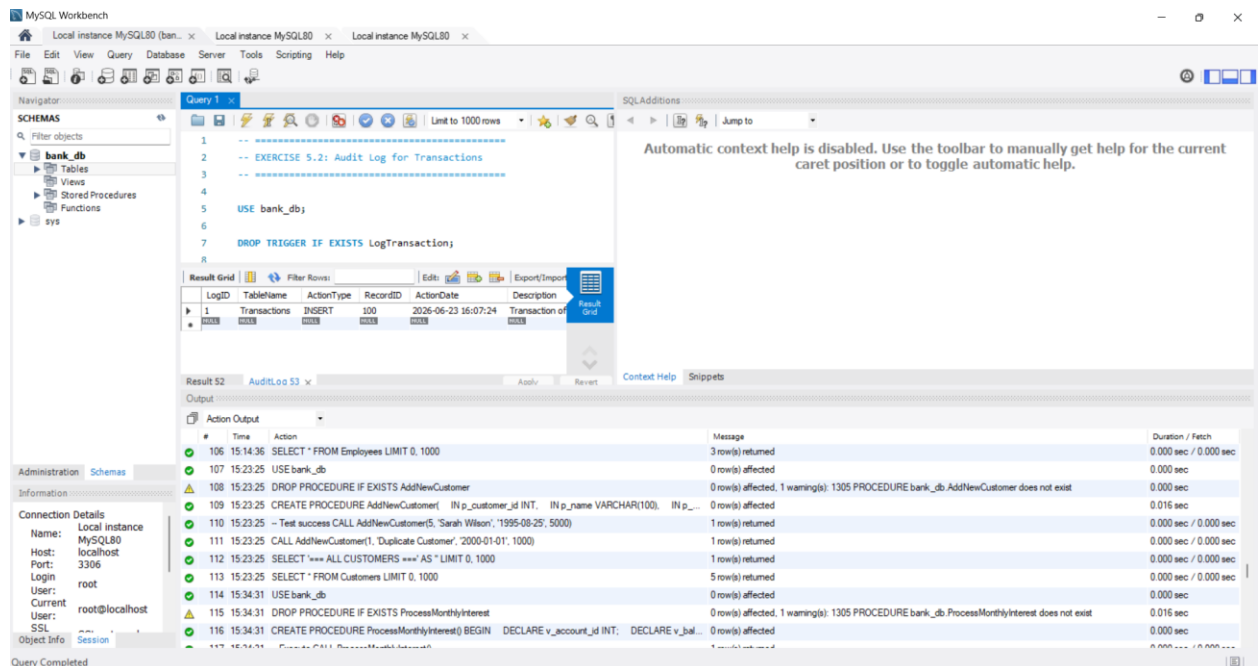
DELIMITER ;

-- Test

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate,
Amount, TransactionType)
VALUES (100, 1, CURDATE(), 50, 'Deposit');
```

```
SELECT '=== AUDIT LOG ===' AS ";
```

```
SELECT * FROM AuditLog ORDER BY LogID DESC LIMIT 5;
```



```

CREATE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
BEGIN
    DECLARE v_balance DECIMAL(15,2);
    DECLARE v_account_exists INT;

    SELECT COUNT(*) INTO v_account_exists
    FROM Accounts
    WHERE AccountID = NEW.AccountID;

    IF v_account_exists = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Account does not exist.';
    END IF;

    IF NEW.TransactionType = 'Withdrawal' THEN
        SELECT Balance INTO v_balance
        FROM Accounts
        WHERE AccountID = NEW.AccountID;

        IF v_balance < NEW.Amount THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Insufficient balance for withdrawal.';
        END IF;
    END IF;

    IF NEW.TransactionType = 'Deposit' AND NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Deposit amount must be positive.';
    END IF;

    IF NEW.TransactionType LIKE 'Transfer%' AND NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transfer amount must be positive.';
    END IF;
END //

DELIMITER ;

```

-- Test successful deposit

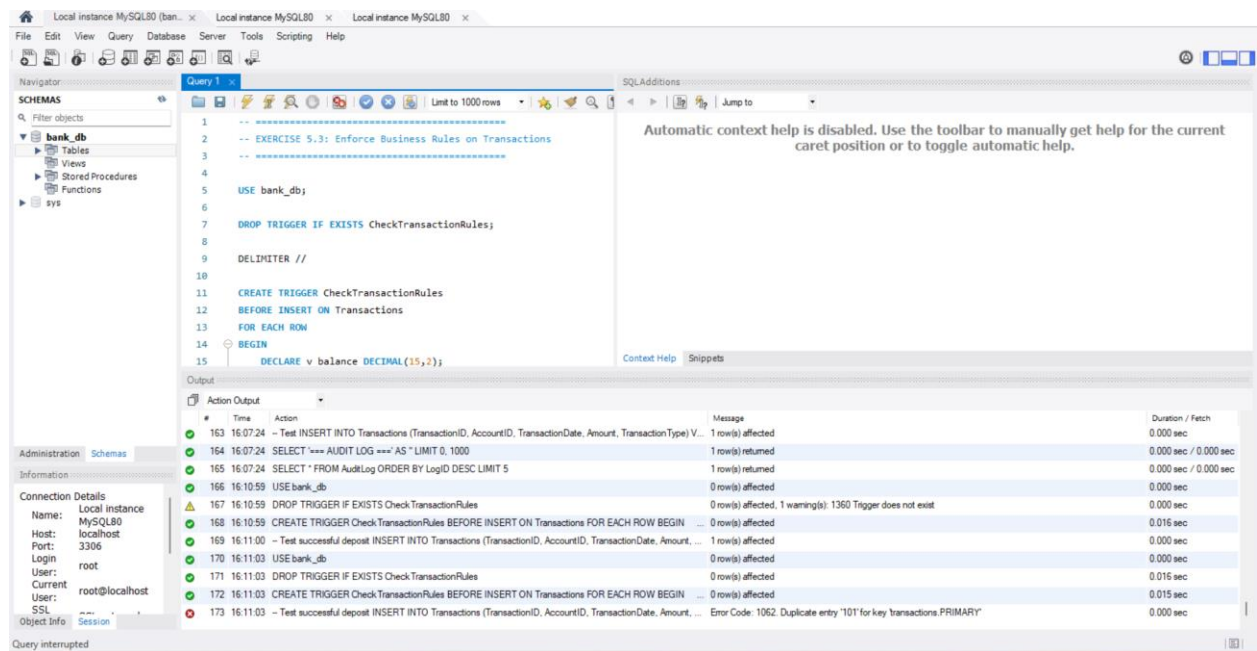
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)

VALUES (101, 1, CURDATE(), 100, 'Deposit');

-- Test failed (negative amount) - This will fail

-- INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)

-- VALUES (102, 1, CURDATE(), -50, 'Deposit');



6. CURSORS

• Scenario 1: Generate Monthly Statements CODE

-- ===== -- EXERCISE 6.1: Generate Monthly Statements --

USE bank_db;

DROP PROCEDURE IF EXISTS GenerateMonthlyStatements;

DELIMITER //

CREATE PROCEDURE GenerateMonthlyStatements() BEGIN DECLARE v_customer_id INT;
DECLARE v_customer_name VARCHAR(100); DECLARE done INT DEFAULT FALSE;

```

DECLARE customer_cursor CURSOR FOR
  SELECT DISTINCT c.CustomerID, c.Name
  FROM Customers c
  INNER JOIN Accounts a ON c.CustomerID = a.CustomerID
  INNER JOIN Transactions t ON a.AccountID = t.AccountID
  WHERE MONTH(t.TransactionDate) = MONTH(CURDATE())
  AND YEAR(t.TransactionDate) = YEAR(CURDATE());

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

DROP TEMPORARY TABLE IF EXISTS temp_statement;

CREATE TEMPORARY TABLE temp_statement (
  StatementLine VARCHAR(500)
);

OPEN customer_cursor;

customer_loop: LOOP
  FETCH customer_cursor INTO v_customer_id, v_customer_name;

  IF done THEN
    LEAVE customer_loop;
  END IF;

  INSERT INTO temp_statement VALUES
    (CONCAT('==== Statement for Customer: ', v_customer_name, ' (ID: ', v_customer_id, ' )
    ====='));

  INSERT INTO temp_statement
  SELECT CONCAT(' ', t.TransactionDate, ' | $', t.Amount, ' | ', t.TransactionType)
  FROM Transactions t
  INNER JOIN Accounts a ON t.AccountID = a.AccountID
  WHERE a.CustomerID = v_customer_id
  AND MONTH(t.TransactionDate) = MONTH(CURDATE())
  AND YEAR(t.TransactionDate) = YEAR(CURDATE());

  INSERT INTO temp_statement VALUES ("");
END LOOP;

```

```
CLOSE customer_cursor;
```

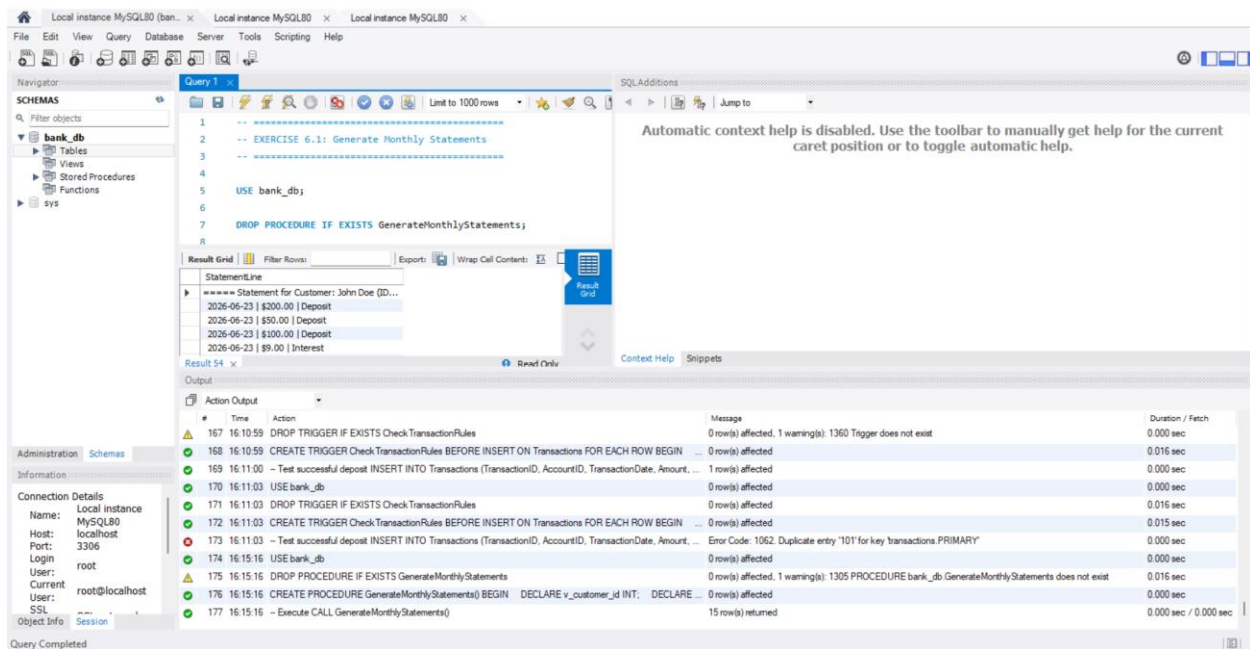
```
SELECT * FROM temp_statement;
```

```
DROP TEMPORARY TABLE temp_statement;
```

```
END //
```

```
DELIMITER ;
```

```
-- Execute CALL GenerateMonthlyStatements();
```



• Scenario 2: Apply Annual Fee CODE

```
-- ===== -- EXERCISE 6.2: Apply  
Annual Fee to All Accounts -- =====
```

```
USE bank_db;
```

```
DROP PROCEDURE IF EXISTS ApplyAnnualFee;
```

```
DELIMITER //
```

```

CREATE PROCEDURE ApplyAnnualFee() BEGIN DECLARE v_account_id INT; DECLARE
v_balance DECIMAL(15,2); DECLARE v_fee DECIMAL(15,2); DECLARE done INT
DEFAULT FALSE;

SET v_fee = 25.00;

DECLARE account_cursor CURSOR FOR
    SELECT AccountID, Balance
    FROM Accounts;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

OPEN account_cursor;

account_loop: LOOP
    FETCH account_cursor INTO v_account_id, v_balance;

    IF done THEN
        LEAVE account_loop;
    END IF;

    IF v_balance >= v_fee THEN
        UPDATE Accounts
        SET Balance = Balance - v_fee,
            LastModified = CURDATE()
        WHERE AccountID = v_account_id;

        INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount,
TransactionType)
            VALUES (FLOOR(RAND()*1000000), v_account_id, CURDATE(), v_fee, 'AnnualFee');
    END IF;
END LOOP;

CLOSE account_cursor;

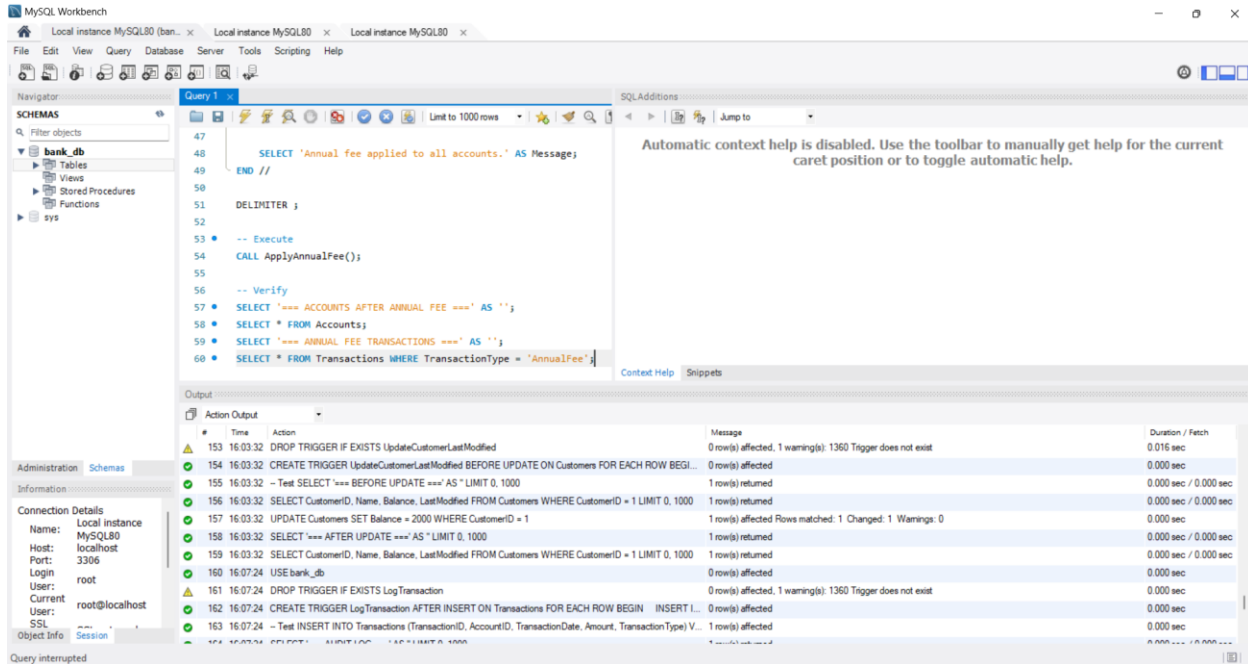
SELECT 'Annual fee applied to all accounts.' AS Message;

END //
DELIMITER ;

```

```
-- Execute CALL ApplyAnnualFee();
```

```
-- Verify SELECT '=== ACCOUNTS AFTER ANNUAL FEE ===' AS "; SELECT * FROM  
Accounts; SELECT '=== ANNUAL FEE TRANSACTIONS ===' AS "; SELECT * FROM  
Transactions WHERE TransactionType = 'AnnualFee';
```



• Scenario 3: Update Loan Interest Rates CODE

```
-- ===== -- EXERCISE 6.3: Update Loan  
Interest Rates Based on Policy -- =====
```

```
USE bank_db;
```

```
DROP PROCEDURE IF EXISTS UpdateLoanInterestRates;
```

```
DELIMITER //
```

```
CREATE PROCEDURE UpdateLoanInterestRates() BEGIN DECLARE v_loan_id INT;  
DECLARE v_customer_dob DATE; DECLARE v_current_rate DECIMAL(5,2); DECLARE  
v_new_rate DECIMAL(5,2); DECLARE v_age INT; DECLARE done INT DEFAULT FALSE;
```

```
DECLARE loan_cursor CURSOR FOR
```

```
    SELECT l.LoanID, c.DOB, l.InterestRate
```

```
    FROM Loans l
```

```
    INNER JOIN Customers c ON l.CustomerID = c.CustomerID;
```

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

```
OPEN loan_cursor;
```

```
loan_loop: LOOP
```

```
    FETCH loan_cursor INTO v_loan_id, v_customer_dob, v_current_rate;
```

```
    IF done THEN
```

```
        LEAVE loan_loop;
```

```
    END IF;
```

```
    SET v_age = TIMESTAMPDIFF(YEAR, v_customer_dob, CURDATE());
```

```
    IF v_age < 30 THEN
```

```
        SET v_new_rate = v_current_rate + 0.25;
```

```
    ELSEIF v_age BETWEEN 30 AND 50 THEN
```

```
        SET v_new_rate = v_current_rate - 0.25;
```

```
    ELSE
```

```
        SET v_new_rate = v_current_rate - 0.50;
```

```
    END IF;
```

```
    IF v_new_rate < 2.00 THEN
```

```
        SET v_new_rate = 2.00;
```

```
    END IF;
```

```
    UPDATE Loans
```

```
    SET InterestRate = v_new_rate
```

```
    WHERE LoanID = v_loan_id;
```

```
END LOOP;
```

```
CLOSE loan_cursor;
```

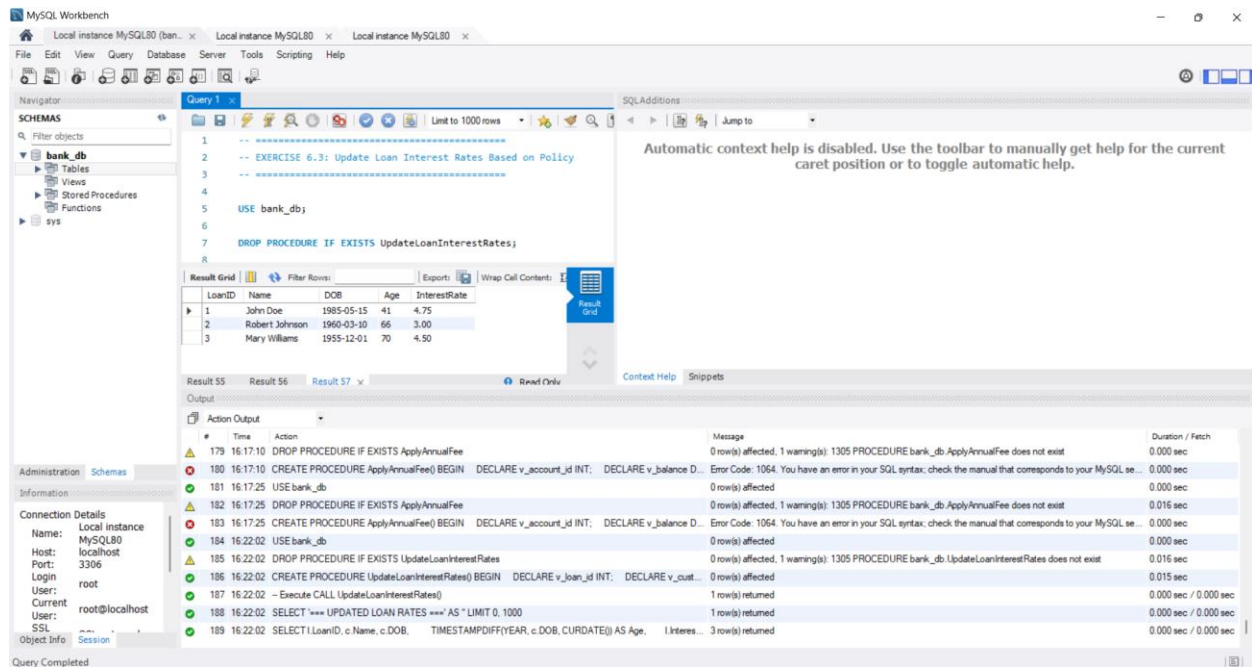
```
SELECT 'Loan interest rates updated based on new policy.' AS Message;
```

```
END //
```

```
DELIMITER ;
```

```
-- Execute CALL UpdateLoanInterestRates();
```

```
-- Verify SELECT '=== UPDATED LOAN RATES ===' AS "; SELECT l.LoanID, c.Name,
c.DOB, TIMESTAMPDIFF(YEAR, c.DOB, CURDATE()) AS Age, l.InterestRate FROM Loans
l INNER JOIN Customers c ON l.CustomerID = c.CustomerID;
```



7. PACKAGES

• Scenario 1: Customer Management Package CODE

```
-- ===== -- EXERCISE 7.1: Customer
Management Package -- =====
```

```
USE bank_db;
```

```
-- Add Customer DROP PROCEDURE IF EXISTS CustomerManagement_AddCustomer;
```

```
DELIMITER //
```

```
CREATE PROCEDURE CustomerManagement_AddCustomer( IN p_customer_id INT, IN
p_name VARCHAR(100), IN p_dob DATE, IN p_balance DECIMAL(15,2) ) BEGIN
DECLARE v_exists INT DEFAULT 0;
```

```
SELECT COUNT(*) INTO v_exists
FROM Customers
WHERE CustomerID = p_customer_id;
```

```
IF v_exists > 0 THEN
```

```

        SELECT CONCAT('Customer ID ', p_customer_id, ' already exists.') AS ErrorMessage;
ELSE
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_customer_id, p_name, p_dob, p_balance, CURDATE());

    SELECT CONCAT('Customer ', p_name, ' added successfully.') AS SuccessMessage;
END IF;

END //

DELIMITER ;

-- Update Customer DROP PROCEDURE IF EXISTS CustomerManagement_UpdateCustomer;
DELIMITER //

CREATE PROCEDURE CustomerManagement_UpdateCustomer( IN p_customer_id INT, IN
p_new_name VARCHAR(100), IN p_new_dob DATE ) BEGIN DECLARE v_exists INT
DEFAULT 0;

SELECT COUNT(*) INTO v_exists
FROM Customers
WHERE CustomerID = p_customer_id;

IF v_exists = 0 THEN
    SELECT CONCAT('Customer ID ', p_customer_id, ' does not exist.') AS ErrorMessage;
ELSE
    UPDATE Customers
    SET Name = COALESCE(p_new_name, Name),
        DOB = COALESCE(p_new_dob, DOB),
        LastModified = CURDATE()
    WHERE CustomerID = p_customer_id;

    SELECT CONCAT('Customer ID ', p_customer_id, ' updated successfully.') AS
SuccessMessage;
END IF;

END //

DELIMITER ;

-- Get Customer Balance DROP FUNCTION IF EXISTS CustomerManagement_GetBalance;

```

DELIMITER //

```
CREATE FUNCTION CustomerManagement_GetBalance(p_customer_id INT) RETURNS  
DECIMAL(15,2) DETERMINISTIC BEGIN DECLARE v_total DECIMAL(15,2);
```

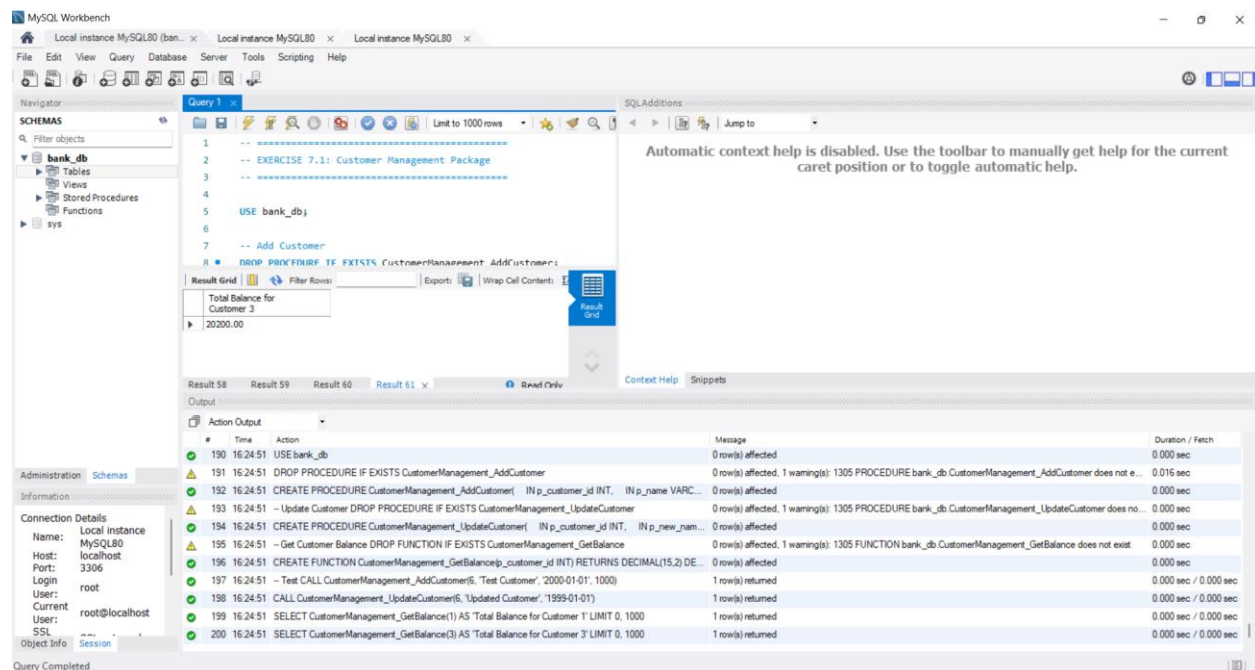
```
SELECT SUM(Balance) INTO v_total  
FROM Accounts  
WHERE CustomerID = p_customer_id;
```

```
RETURN COALESCE(v_total, 0);
```

END //

DELIMITER ;

```
-- Test CALL CustomerManagement_AddCustomer(6, 'Test Customer', '2000-01-01', 1000);  
CALL CustomerManagement_UpdateCustomer(6, 'Updated Customer', '1999-01-01'); SELECT  
CustomerManagement_GetBalance(1) AS 'Total Balance for Customer 1'; SELECT  
CustomerManagement_GetBalance(3) AS 'Total Balance for Customer 3';
```



• Scenario 2: Employee Management Package CODE

```
-- ===== -- EXERCISE 7.2: Employee  
Management Package -- =====
```

```

USE bank_db;

-- Hire Employee DROP PROCEDURE IF EXISTS EmployeeManagement_HireEmployee;

DELIMITER //

CREATE PROCEDURE EmployeeManagement_HireEmployee( IN p_employee_id INT, IN
p_name VARCHAR(100), IN p_position VARCHAR(50), IN p_salary DECIMAL(15,2), IN
p_department VARCHAR(50) ) BEGIN DECLARE v_exists INT DEFAULT 0;

SELECT COUNT(*) INTO v_exists
FROM Employees
WHERE EmployeeID = p_employee_id;

IF v_exists > 0 THEN
    SELECT CONCAT('Employee ID ', p_employee_id, ' already exists.') AS ErrorMessage;
ELSE
    INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
    VALUES (p_employee_id, p_name, p_position, p_salary, p_department, CURDATE());

    SELECT CONCAT('Employee ', p_name, ' hired successfully.') AS SuccessMessage;
END IF;

END //

DELIMITER ;

-- Update Employee DROP PROCEDURE IF EXISTS
EmployeeManagement_UpdateEmployee;

DELIMITER //

CREATE PROCEDURE EmployeeManagement_UpdateEmployee( IN p_employee_id INT, IN
p_new_name VARCHAR(100), IN p_new_position VARCHAR(50), IN p_new_salary
DECIMAL(15,2), IN p_new_department VARCHAR(50) ) BEGIN DECLARE v_exists INT
DEFAULT 0;

SELECT COUNT(*) INTO v_exists
FROM Employees
WHERE EmployeeID = p_employee_id;

IF v_exists = 0 THEN
    SELECT CONCAT('Employee ID ', p_employee_id, ' does not exist.') AS ErrorMessage;
ELSE

```

```

UPDATE Employees
SET Name = COALESCE(p_new_name, Name),
    Position = COALESCE(p_new_position, Position),
    Salary = COALESCE(p_new_salary, Salary),
    Department = COALESCE(p_new_department, Department)
WHERE EmployeeID = p_employee_id;

SELECT CONCAT('Employee ID ', p_employee_id, ' updated successfully.') AS
SuccessMessage;
END IF;

END //

DELIMITER ;

-- Calculate Annual Salary DROP FUNCTION IF EXISTS
EmployeeManagement_AnnualSalary;

DELIMITER //

CREATE FUNCTION EmployeeManagement_AnnualSalary(p_employee_id INT) RETURNS
DECIMAL(15,2) DETERMINISTIC BEGIN DECLARE v_salary DECIMAL(15,2);

SELECT Salary INTO v_salary
FROM Employees
WHERE EmployeeID = p_employee_id;

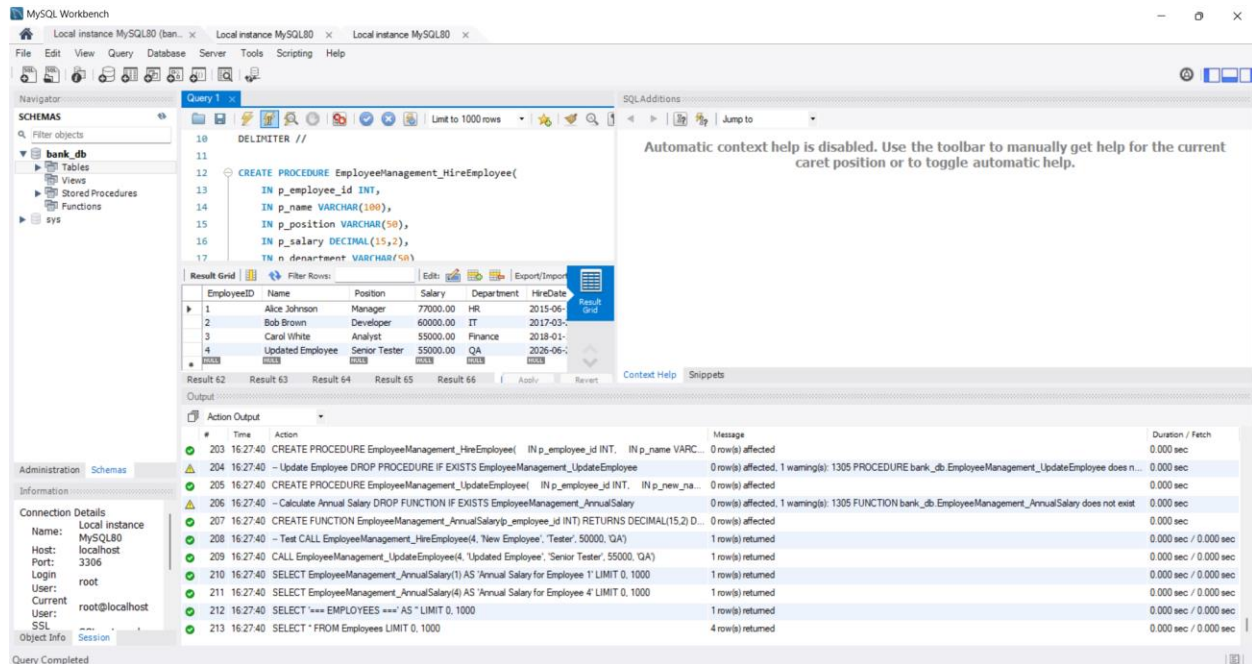
RETURN v_salary * 12;

END //

DELIMITER ;

-- Test CALL EmployeeManagement_HireEmployee(4, 'New Employee', 'Tester', 50000, 'QA');
CALL EmployeeManagement_UpdateEmployee(4, 'Updated Employee', 'Senior Tester', 55000,
'QA'); SELECT EmployeeManagement_AnnualSalary(1) AS 'Annual Salary for Employee 1';
SELECT EmployeeManagement_AnnualSalary(4) AS 'Annual Salary for Employee 4'; SELECT
'=== EMPLOYEES ===' AS ""; SELECT * FROM Employees;

```



- Scenario 3: Account Operations Package **CODE**

-- ===== -- EXERCISE 7.3: Account Operations Package -- =====

USE bank_db;

-- Open Account DROP PROCEDURE IF EXISTS AccountOperations_OpenAccount;

DELIMITER //

```
CREATE PROCEDURE AccountOperations_OpenAccount( IN p_account_id INT, IN
p_customer_id INT, IN p_account_type VARCHAR(20), IN p_initial_balance
DECIMAL(15,2) ) BEGIN DECLARE v_customer_exists INT DEFAULT 0; DECLARE
v_account_exists INT DEFAULT 0;
```

```
SELECT COUNT(*) INTO v_customer_exists
FROM Customers
WHERE CustomerID = p_customer_id;
```

```
IF v_customer_exists = 0 THEN
```

```
    SELECT CONCAT('Customer ID ', p_customer_id, ' does not exist.') AS ErrorMessage;
```

```
ELSE
```

```
    SELECT COUNT(*) INTO v_account_exists
    FROM Accounts
```

```

WHERE AccountID = p_account_id;

IF v_account_exists > 0 THEN
    SELECT CONCAT('Account ID ', p_account_id, ' already exists.') AS ErrorMessage;
ELSE
    INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)
    VALUES (p_account_id, p_customer_id, p_account_type, p_initial_balance,
CURDATE());

    SELECT CONCAT('Account ', p_account_id, ' opened successfully.') AS SuccessMessage;
END IF;
END IF;

END //

DELIMITER ;

-- Close Account DROP PROCEDURE IF EXISTS AccountOperations_CloseAccount;

DELIMITER //

CREATE PROCEDURE AccountOperations_CloseAccount( IN p_account_id INT ) BEGIN
DECLARE v_balance DECIMAL(15,2); DECLARE v_exists INT DEFAULT 0;

SELECT COUNT(*) INTO v_exists
FROM Accounts
WHERE AccountID = p_account_id;

IF v_exists = 0 THEN
    SELECT CONCAT('Account ID ', p_account_id, ' does not exist.') AS ErrorMessage;
ELSE
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_account_id;

    IF v_balance > 0 THEN
        SELECT CONCAT('Account has balance of $', v_balance,
            ' . Cannot close account with remaining balance.') AS ErrorMessage;
    ELSE
        DELETE FROM Accounts
        WHERE AccountID = p_account_id;
    
```

```
        SELECT CONCAT('Account ', p_account_id, ' closed successfully.') AS SuccessMessage;
    END IF;
END IF;
```

```
END //
```

```
DELIMITER ;
```

```
-- Get Total Balance DROP FUNCTION IF EXISTS AccountOperations_GetTotalBalance;
```

```
DELIMITER //
```

```
CREATE FUNCTION AccountOperations_GetTotalBalance(p_customer_id INT) RETURNS
DECIMAL(15,2) DETERMINISTIC BEGIN DECLARE v_total DECIMAL(15,2);
```

```
SELECT SUM(Balance) INTO v_total
FROM Accounts
WHERE CustomerID = p_customer_id;
```

```
RETURN COALESCE(v_total, 0);
```

```
END //
```

```
DELIMITER ;
```

```
-- Test SELECT '=== BEFORE OPENING ACCOUNT ===' AS "; SELECT * FROM
Accounts;
```

```
CALL AccountOperations_OpenAccount(5, 1, 'Savings', 500); SELECT
AccountOperations_GetTotalBalance(1) AS 'Total Balance for Customer 1';
```

```
SELECT '=== AFTER OPENING ACCOUNT ===' AS "; SELECT * FROM Accounts;
```

MySQL Workbench

Local instance MySQL80 (ban... Local instance MySQL80 Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

bank_db

Tables

Views

Stored Procedures

Functions

sys

Query 1

Limit to 1000 rows

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

AccountID	CustomerID	AccountType	Balance	LastModified
1	1	Savings	909.00	2026-06-23
2	2	Checking	1600.00	2026-06-23
3	3	Savings	20200.00	2026-06-23
4	4	Checking	5000.00	2026-06-23
5	1	Savings	500.00	2026-06-23

Result 68 Accounts 69 Result 70 Result 71 Result 72 Apply Revert Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
216	16:29:40	CREATE PROCEDURE AccountOperations_OpenAccount(IN p_account_id INT, IN p_customer_id INT...	0 row(s) affected	0.016 sec
217	16:29:40	-- Close Account DROP PROCEDURE IF EXISTS AccountOperations_CloseAccount	0 row(s) affected, 1 warning(s): 1305 PROCEDURE bank_db.AccountOperations_CloseAccount does not exist	0.000 sec
218	16:29:40	CREATE PROCEDURE AccountOperations_CloseAccount(IN p_account_id INT) BEGIN DECLARE v_...	0 row(s) affected	0.015 sec
219	16:29:40	-- Get Total Balance DROP FUNCTION IF EXISTS AccountOperations_GetTotalBalance	0 row(s) affected, 1 warning(s): 1305 FUNCTION bank_db.AccountOperations_GetTotalBalance does not exist	0.000 sec
220	16:29:40	CREATE FUNCTION AccountOperations_GetTotalBalance(p_customer_id INT) RETURNS DECIMAL(15,2) D...	0 row(s) affected	0.016 sec
221	16:29:40	-- Test SELECT '==== BEFORE OPENING ACCOUNT ===' AS 'LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
222	16:29:40	SELECT * FROM Accounts LIMIT 0, 1000	4 row(s) returned	0.015 sec / 0.000 sec
223	16:29:40	CALL AccountOperations_OpenAccount(5, 1, 'Savings', 500)	1 row(s) returned	0.000 sec / 0.000 sec
224	16:29:40	SELECT AccountOperations_GetTotalBalance(1) AS 'Total Balance for Customer 1' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
225	16:29:41	SELECT '==== AFTER OPENING ACCOUNT ===' AS 'LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
226	16:29:41	SELECT * FROM Accounts LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec

Administration Schemas

Information

Connection Details

Name: Local instance
Host: MySQL80
Port: localhost
3306
Login: root
User: root
Current User: root@localhost
SSL: -- -- --
Object Info: Session

Query Completed